

API Documentation

Health Check Endpoints

API Gateway Microservice:

Endpoint: /health
Method: GET

Success Response:
{msg: 'UP'}

Auth Microservice:

Endpoint: /auth/test/health
Method: GET

Success Response:
{ "status": "UP" }

Leave-manager Microservice:

Endpoint: /leave-manager/test/health
Method: GET

Success Response:
{ "msg": "UP" }

Notifications Microservice:

Endpoint: /leave-manager/test/notifications/health
Method: GET

Success Response:
{ "msg": "UP" }

Auth Microservice Endpoints

1] **Endpoint:** /auth/login
Method: POST

Request Body:

```
{"employeeId": string, "password": string }
```

E.g. {"employeeId": "EMP-001", "password": "\$2b\$10\$eI29y3xG7" }

Success Response:

Code: 200

JWT_TOKEN_AS_VALUE

e.g.

```
eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbXBsb3llZUkljoiRU1QLTAwMSIsInJvbGUiOiJFTVBMT1lFRSIsImIhdCI6MTc4MDg5NTA4OSwiZXhwIjoxNzgwODk4Njg5fQ.AvFBB6r2yWAQACJs1kxGSksGSG02dXJ7_Wu3e2VGZlyLU0XovhxqO318kuej7SA69PKP2mYw0kd_btPlqZLHFxJgrH6dAzNgHotECbXL2L-aqIRijTlkD7BzbLhydnt3kySN1eAQVDTh6eUAd_KyKox-2xQhtPfc-UpQNoY-Niv_lvrM4PMQqQWds_AUwJ7sM7IJ8Hw1ODBAawSGNpnSDHM7GcD4Xi9B2gf31vQlnSgBWxgv00scNVVHBMzxrT5FFSZL-ViBVm0Bmy_CeeQqqFg9SljSNpE7JpbriT3P0HvsJ239ouNmKVU5fK_P_z3kRbQDo6FzbDfeZnKzn4qqg
```

Payload of JWT:

```
{  
  "employeeId": "EMP-001",  
  "role": "EMPLOYEE",  
  "iat": 1780894072,  
  "exp": 1780897672  
}
```

2] **Endpoint:** /auth/jwks.json
Method: GET

Success Response:

Code: 200

```
{
  "keys": [
    {
      "kty": "RSA",
      "use": "sig",
      "alg": "RS256",
      "kid": "auth-key-v1",
      "n": "...",
      "e": "..."
    }
  ]
}
```

Leave-Manager Microservice Endpoints

- 1] **Endpoint:** /leave-manager/view-balance
Method: GET

Request Header:

Authorization: Bearer JWT_TOKEN

Allowed Roles: EMPLOYEE, MANAGER**Query Params:**

- type: CASUAL | SICK | PRIVILEGE (Optional)

Success Response:

Code: 200

```
[
  {
    "type": "CASUAL",
    "balance": number
  },
  {
    "type": "PRIVILEGE",
    "balance": number
  },
  {

```

```
    "type": "SICK",  
    "balance": number  
  }  
]
```

E.g.

```
[  
  {  
    "type": "CASUAL",  
    "balance": 12  
  },  
  {  
    "type": "PRIVILEGE",  
    "balance": 15  
  },  
  {  
    "type": "SICK",  
    "balance": 9  
  }  
]
```

With type: CASUAL | SICK | PRIVILEGE

E.g. /leave-manager/view-balance?type=CASUAL

```
[  
  {  
    "type": "CASUAL",  
    "balance": 12  
  }  
]
```

Failure Scenarios:

- **Invalid/Expired JWT token:**

Code: 401

Sample Response:

```
{  
  "statusCode": 401,  
  "timestamp": "2026-06-08T06:38:07.105Z",  
  "message": {  
    "statusCode": 401,  
    "timestamp": "2026-06-08T06:38:07.091Z",  
    "message": {  
      "message": "Unauthorized",  
      "statusCode": 401  
    }  
  }  
}
```

```
}  
}  
}
```

- **Incorrect query parameter: type**

Code: 400

Sample Response:

```
{  
  "statusCode": 400,  
  "timestamp": "2026-06-08T06:44:26.203Z",  
  "message": {  
    "statusCode": 400,  
    "timestamp": "2026-06-08T06:44:26.180Z",  
    "message": {  
      "message": [  
        "Invalid leave type. Must be CASUAL, SICK, or PRIVILEGE"  
      ],  
      "error": "Bad Request",  
      "statusCode": 400  
    }  
  }  
}
```

2] **Endpoint:** /leave-manager/view-balance/employee?empld=EmployeeId

E.g. /leave-manager/view-balance/employee?empld=EMP-001

Method: GET

QueryParam: empld (Mandatory)

Request Header:

Authorization: Bearer JWT_TOKEN

Allowed Roles: MANAGER

Success Response:

Code: 200

```
[
  {
    "type": "CASUAL",
    "balance": number
  },
  {
    "type": "PRIVILEGE",
    "balance": number
  },
  {
    "type": "SICK",
    "balance": number
  }
]
```

Sample Response:

```
[
  {
    "type": "CASUAL",
    "balance": 12
  },
  {
    "type": "PRIVILEGE",
    "balance": 15
  },
  {
    "type": "SICK",
    "balance": 9
  }
]
```

Failure Scenarios:

- **No empld param value**

Code: 404

Sample Response:

```
{
  "statusCode": 404,
  "timestamp": "2026-06-08T07:13:31.730Z",
  "message": {
    "statusCode": 404,
    "timestamp": "2026-06-08T07:13:31.728Z",
    "message": {
      "message": "User Not Valid",
      "error": "Not Found",
      "statusCode": 404
    }
  }
}
```

```
}  
}  
}
```

- **Wrong empld value or employeeId does not have the user as the reporting manager, (Jwt token passed is not of the reportingManager or passed empld)**

Code: 403

Sample Response:

```
{  
  "statusCode": 403,  
  "timestamp": "2026-06-08T07:10:13.712Z",  
  "message": {  
    "statusCode": 403,  
    "timestamp": "2026-06-08T07:10:13.710Z",  
    "message": {  
      "message": "You are not authorized to view this employee's balance.",  
      "error": "Forbidden",  
      "statusCode": 403  
    }  
  }  
}
```

- **Role is not of Manager (Jwt token passed is of employee with role as EMPLOYEE).**

Code: 403

Sample Response:

```
{  
  "statusCode": 403,  
  "timestamp": "2026-06-08T07:16:41.798Z",  
  "message": {  
    "statusCode": 403,  
    "timestamp": "2026-06-08T07:16:41.791Z",  
    "message": {  
      "message": "You do not have the required role",  
      "error": "Forbidden",  
      "statusCode": 403  
    }  
  }  
}
```

3] **Endpoint:** /leave-manager/leaves

Method: Get

Request Header:

Authorization: Bearer JWT_TOKEN

Query Params: (all are optional, in case filtration or pagination is needed)

- page: default 1
- limit: default 10, max: 100
- type: CASUAL | SICK | PRIVILEGE
- status: PENDING | APPROVED | REJECTED | CANCELLED

Allowed Roles: EMPLOYEE, MANAGER

Success Response:

Code: 200

```
{
  "data": [
    {
      "id": LEAVE_ID,
      "employeeId": EmployeeId,
      "type": CASUAL | SICK | PRIVILEGE,
      "startDate": start_date_of_leave,
      "endDate": end_date_of_leave,
      "status": APPROVED | REJECTED | CANCELLED | PENDING,
      "reportingManagerId": EmployeeId_of_Manager
    }
  ],
  "pagination": {
    "total": TOTAL_ITEMS,
    "page": 1,
    "limit": 10,
    "totalPages": 1
  }
}
```

Sample Success Response:

```
{
  "data": [
    {
      "id": "cmq3fe34t000301te8czdpa2h",
      "employeeId": "EMP-001",
      "type": "SICK",
      "startDate": "2026-06-14T00:00:00.000Z",
      "endDate": "2026-06-16T00:00:00.000Z",
      "status": "REJECTED",
      "reportingManagerId": "MGR-002"
    }
  ]
}
```



```

    },
    {
      "id": "cmq3fgghl000101pjplv2clz5",
      "employeeId": "EMP-001",
      "type": "SICK",
      "startDate": "2026-06-14T00:00:00.000Z",
      "endDate": "2026-06-16T00:00:00.000Z",
      "status": "CANCELLED",
      "reportingManagerId": "MGR-002"
    },
    {
      "id": "cmq3ei6ws000101tezqa82ytt",
      "employeeId": "EMP-001",
      "type": "SICK",
      "startDate": "2026-06-13T00:00:00.000Z",
      "endDate": "2026-06-13T00:00:00.000Z",
      "status": "APPROVED",
      "reportingManagerId": "MGR-002"
    },
    {
      "id": "cmq3fbd17000201te2u7dsuje",
      "employeeId": "EMP-001",
      "type": "SICK",
      "startDate": "2026-06-12T00:00:00.000Z",
      "endDate": "2026-06-12T00:00:00.000Z",
      "status": "APPROVED",
      "reportingManagerId": "MGR-002"
    }
  ],
  "pagination": {
    "total": 4,
    "page": 1,
    "limit": 10,
    "totalPages": 1
  }
}

```

Failure Scenarios:

- **Limit Too High (more than 100)**

Code: 400

Sample Response:

```

{
  "statusCode": 400,
  "timestamp": "2026-06-08T07:59:45.760Z",
  "message": {
    "statusCode": 400,

```

```

        "timestamp": "2026-06-08T07:59:45.756Z",
        "message": {
            "message": [
                "Limit cannot exceed 100 per request"
            ],
            "error": "Bad Request",
            "statusCode": 400
        }
    }
}

```

- **Wrong Type:**

Code: 400

Sample Response:

```

{
    "statusCode": 400,
    "timestamp": "2026-06-08T07:35:48.242Z",
    "message": {
        "statusCode": 400,
        "timestamp": "2026-06-08T07:35:48.237Z",
        "message": {
            "message": [
                "Invalid leave type. Must be CASUAL, SICK, or PRIVILEGE"
            ],
            "error": "Bad Request",
            "statusCode": 400
        }
    }
}

```

- **Wrong status:**

Code: 400

Sample Response:

```

{
    "statusCode": 400,
    "timestamp": "2026-06-08T07:35:37.954Z",
    "message": {
        "statusCode": 400,
        "timestamp": "2026-06-08T07:35:37.950Z",
        "message": {
            "message": [
                "Invalid status. Must be PENDING, APPROVED, or REJECTED."
            ],
            "error": "Bad Request",
            "statusCode": 400
        }
    }
}

```

```
}
```

- **Wrong/Expired Jwt**

Code: 401

Sample Response:

```
{
  "statusCode": 401,
  "timestamp": "2026-06-08T08:07:28.459Z",
  "message": {
    "statusCode": 401,
    "timestamp": "2026-06-08T08:07:28.455Z",
    "message": {
      "message": "Unauthorized",
      "statusCode": 401
    }
  }
}
```

4] **Endpoint:** /leave-manager/leaves/employees

Method: Get

Request Header:

Authorization: Bearer JWT_TOKEN

Query Params: (all are optional, in case filtration or pagination is needed)

- page: default 1
- limit: default 10, max: 100
- type: CASUAL | SICK | PRIVILEGE
- status: PENDING | APPROVED | REJECTED | CANCELLED
- empld: string (employeeId)

Allowed Roles: MANAGER

Success Response:

200

```
{
  "data": [
    {
      "id": LEAVE_ID,
      "employeeId": EmployeeId,
      "type": CASUAL | SICK | PRIVILEGE,
      "startDate": start_date_of_leave,
      "endDate": end_date_of_leave,
    }
  ]
}
```

```

        "status": APPROVED | REJECTED | CANCELLED | PENDING,
        "reportingManagerId": EmployeeId_of_Manager
    }
],
"pagination": {
    "total": TOTAL_ITEMS,
    "page": 1,
    "limit": 10,
    "totalPages": 1
}
}

```

Sample Response:

```

{
    "data": [
        {
            "id": "cmq3fe34t000301te8czdpa2h",
            "employeeId": "EMP-001",
            "type": "SICK",
            "startDate": "2026-06-14T00:00:00.000Z",
            "endDate": "2026-06-16T00:00:00.000Z",
            "status": "REJECTED",
            "reportingManagerId": "MGR-002"
        }
    ],
    "pagination": {
        "total": 1,
        "page": 1,
        "limit": 10,
        "totalPages": 1
    }
}

```

Failure Scenarios:

- **Role Not as Manager.**

```

{
    "statusCode": 403,
    "timestamp": "2026-06-08T08:13:48.254Z",
    "message": {
        "statusCode": 403,
        "timestamp": "2026-06-08T08:13:48.253Z",
        "message": "You do not have the required role",
        "error": "Forbidden",
        "statusCode": 403
    }
}

```

```
}  
}  
}
```

- **Limit Too High (more than 100)**

Code: 400

Sample Response:

```
{  
  "statusCode": 400,  
  "timestamp": "2026-06-08T07:59:45.760Z",  
  "message": {  
    "statusCode": 400,  
    "timestamp": "2026-06-08T07:59:45.756Z",  
    "message": {  
      "message": [  
        "Limit cannot exceed 100 per request"  
      ],  
      "error": "Bad Request",  
      "statusCode": 400  
    }  
  }  
}
```

- **Wrong Type:**

Code: 400

Sample Response:

```
{  
  "statusCode": 400,  
  "timestamp": "2026-06-08T07:35:48.242Z",  
  "message": {  
    "statusCode": 400,  
    "timestamp": "2026-06-08T07:35:48.237Z",  
    "message": {  
      "message": [  
        "Invalid leave type. Must be CASUAL, SICK, or PRIVILEGE"  
      ],  
      "error": "Bad Request",  
      "statusCode": 400  
    }  
  }  
}
```

- **Wrong status:**

Code: 400

Sample Response:

```
{
  "statusCode": 400,
  "timestamp": "2026-06-08T07:35:37.954Z",
  "message": {
    "statusCode": 400,
    "timestamp": "2026-06-08T07:35:37.950Z",
    "message": {
      "message": [
        "Invalid status. Must be PENDING, APPROVED, or REJECTED."
      ],
      "error": "Bad Request",
      "statusCode": 400
    }
  }
}
```

- **Wrong/Expired Jwt**

Code: 401

Sample Response:

```
{
  "statusCode": 401,
  "timestamp": "2026-06-08T08:07:28.459Z",
  "message": {
    "statusCode": 401,
    "timestamp": "2026-06-08T08:07:28.455Z",
    "message": {
      "message": "Unauthorized",
      "statusCode": 401
    }
  }
}
```

5] **Endpoint:** /leave-manager/leaves/apply

Method: POST

Request Header:

Authorization: Bearer JWT_TOKEN

Request Body:

```
{ "type": SICK | CASUAL | PRIVILEGE, "startDate": TODAY OR FUTURE,
  "endDate": TODAY OR FUTURE, "reason": "MUST BE UNDER 100 characters"
}
```

- type (Mandatory)
- startDate(Mandatory)
- endDate (Mandatory)
- reason (Optional)

Sample Body:

```
{ "type": "SICK", "startDate": "2026-06-8", "endDate": "2026-06-8" }
```

Allowed Roles: EMPLOYEE, MANAGER

Point to Remember: cannot apply for a leave that clashes with the dates of an already applied leave.

Success Response:

```
{
  "id": LEAVE_ID,
  "employeeId": APPLICANT_ID,
  "type": SICK | CASUAL | PRIVILEGE,
  "startDate": start_date_of_leave,
  "endDate": end_date_of_leave,
  "status": PENDING | CANCELLED | APPROVED | REJECTED,
  "reportingManagerId": reporting_manger_id,
  "applyreason": ""
  "rejectreason":""
}
```

Failure Scenarios:

- **Start Date is in the Past**

Sample Response:

```
{
  "statusCode": 400,
  "timestamp": "2026-06-08T08:35:55.447Z",
  "message": {
    "statusCode": 400,
    "timestamp": "2026-06-08T08:35:55.443Z",
    "message": {
      "message": "Start date cannot be in the past",
      "error": "Bad Request",
      "statusCode": 400
    }
  }
}
```

- **End Date is set before start date**

Sample Response:

```
{
  "statusCode": 400,
  "timestamp": "2026-06-08T08:36:39.737Z",
  "message": {
    "statusCode": 400,
    "timestamp": "2026-06-08T08:36:39.734Z",
    "message": {
      "message": "End Date is before start date",
      "error": "Bad Request",
      "statusCode": 400
    }
  }
}
```

- **If the applicant is trying to apply for a leave that is clashing with an already approved leave.**

Sample Response:

Code: 409 Conflict

```
{
  "statusCode": 409,
  "timestamp": "2026-06-08T08:38:08.022Z",
  "message": {
    "statusCode": 409,
    "timestamp": "2026-06-08T08:38:08.018Z",
    "message": {
      "message": "Date range clashes with an existing approved leave",
      "error": "Conflict",
      "statusCode": 409
    }
  }
}
```

- **Wrong/Expired Jwt**

Code: 401

Sample Response:

```
{
  "statusCode": 401,
  "timestamp": "2026-06-08T08:07:28.459Z",
  "message": {
    "statusCode": 401,
    "timestamp": "2026-06-08T08:07:28.455Z",
    "message": {
```



```

        "message": "Unauthorized",
        "statusCode": 401
    }
}
}

```

- **Wrong type passed in body**

Code: 400

Sample Response:

```

{
  "statusCode": 400,
  "timestamp": "2026-06-08T08:40:11.922Z",
  "message": {
    "statusCode": 400,
    "timestamp": "2026-06-08T08:40:11.917Z",
    "message": {
      "message": [
        "type must be one of the following values: CASUAL, SICK, PRIVILEGE"
      ],
      "error": "Bad Request",
      "statusCode": 400
    }
  }
}

```

- **If the duration between start and end date exceeds the available balance of the applicant by type (duration includes only business days, Saturday/Sunday Not considered).**

Sample Response:

Code: 400

```

{
  "statusCode": 400,
  "timestamp": "2026-06-08T08:41:15.410Z",
  "message": {
    "statusCode": 400,
    "timestamp": "2026-06-08T08:41:15.405Z",
    "message": {
      "message": "Insufficient SICK balance. Need 131 days.",
      "error": "Bad Request",
      "statusCode": 400
    }
  }
}

```

- **If reason is more than 100 characters**

Sample Response:

```
{
  "statusCode": 400,
  "timestamp": "2026-06-08T13:03:06.979Z",
  "message": {
    "statusCode": 400,
    "timestamp": "2026-06-08T13:03:06.974Z",
    "message": {
      "message": [
        "Reason must be under 100 characters"
      ],
      "error": "Bad Request",
      "statusCode": 400
    }
  }
}
```

6] **Endpoint:** /leave-manager/leaves/employees/approve

Method: POST

QueryParams:

- leaveId (mandatory)

E.g.

/leave-manager/leaves/employees/approve?leaveId=cmq4yc6ns000001pj1oilip5t

Request Header:

Authorization: Bearer JWT_TOKEN

Allowed Roles: MANAGER

Success Response:

```
{
  "id": LEAVE_ID,
  "employeeId": APPLICANT_ID,
  "type": SICK | CASUAL | PRIVILEGE,
  "startDate": start_date_of_leave,
  "endDate": end_date_of_leave,
  "status": APPROVED,
  "reportingManagerId": reporting_manger_id
}
```

Code: 200

Sample Response:

```
{
  "id": "cmq4yc6ns000001pj1oilip5t",
  "employeeId": "EMP-001",
  "type": "SICK",
  "startDate": "2026-06-08T00:00:00.000Z",
  "endDate": "2026-06-08T00:00:00.000Z",
  "status": "APPROVED",
  "reportingManagerId": "MGR-002"
  "applyreason": ""
  "rejectreason": ""
}
```

Failure Scenarios:

- If the LeaveId is not in PENDING state

Code: 400

```
{
  "statusCode": 400,
  "timestamp": "2026-06-08T10:37:57.459Z",
  "message": {
    "statusCode": 400,
    "timestamp": "2026-06-08T10:37:57.453Z",
    "message": {
      "message": "Request is not pending",
      "error": "Bad Request",
      "statusCode": 400
    }
  }
}
```

- If leave Id empty or not passed in the query param.

Sample Response:

Code: 400

```
{
  "statusCode": 400,
  "timestamp": "2026-06-08T08:44:48.333Z",
  "message": {
    "statusCode": 400,
    "timestamp": "2026-06-08T08:44:48.331Z",
    "message": {
```

```

        "message": [
            "leaveld must be a string",
            "leaveld should not be empty"
        ],
        "error": "Bad Request",
        "statusCode": 400
    }
}

```

- **If leaveld passed does not exist**

Sample Response:
Code 404

```

{
  "statusCode": 404,
  "timestamp": "2026-06-08T10:41:16.470Z",
  "message": {
    "statusCode": 404,
    "timestamp": "2026-06-08T10:41:16.468Z",
    "message": {
      "message": "Leave request not found",
      "error": "Not Found",
      "statusCode": 404
    }
  }
}

```

- **If the user trying to approve the leave does not have role as MANAGER**

Sample Response:
Code: 403

```

{
  "statusCode": 403,
  "timestamp": "2026-06-08T08:44:06.029Z",
  "message": {
    "statusCode": 403,
    "timestamp": "2026-06-08T08:44:06.027Z",
    "message": {
      "message": "You do not have the required role",
      "error": "Forbidden",
      "statusCode": 403
    }
  }
}

```

- **Wrong/Expired Jwt**

Code: 401

Sample Response:

```
{
  "statusCode": 401,
  "timestamp": "2026-06-08T08:07:28.459Z",
  "message": {
    "statusCode": 401,
    "timestamp": "2026-06-08T08:07:28.455Z",
    "message": {
      "message": "Unauthorized",
      "statusCode": 401
    }
  }
}
```

- **If the leave balance of that type of leave is not enough to apply for the request.**

```
{
  "statusCode": 400,
  "timestamp": "2026-06-08T13:03:06.979Z",
  "message": {
    "statusCode": 400,
    "timestamp": "2026-06-08T13:03:06.974Z",
    "message": {
      "message": [
        "Insufficient balance to approve this request"
      ],
      "error": "Bad Request",
      "statusCode": 400
    }
  }
}
```

7] **Endpoint:** leave-manager/leaves/employees/reject

Method: POST

Query Param:

- leaveld (mandatory)

E.g.

leave-manager/leaves/employees/reject?leaveld=cmq4yc6ns000001pj1oilip5t

Request Header:

Authorization: Bearer JWT_TOKEN

Request Body:

{reason: 'Must be under 100 characters'}

- reason (Optional)

Allowed Roles: MANAGER

Success Response:

```
{
  "id": LEAVE_ID,
  "employeeId": APPLICANT_ID,
  "type": SICK | CASUAL | PRIVILEGE,
  "startDate": start_date_of_leave,
  "endDate": end_date_of_leave,
  "status": REJECTED,
  "reportingManagerId": reporting_manger_id
  "applyreason": ""
  "rejectreason": ""
}
```

Code: 200

Sample Response:

```
{
  "id": "cmq4yc6ns000001pj1oilip5t",
  "employeeId": "EMP-001",
  "type": "SICK",
  "startDate": "2026-06-08T00:00:00.000Z",
  "endDate": "2026-06-08T00:00:00.000Z",
  "status": "REJECTED",
  "reportingManagerId": "MGR-002"
  "applyreason": ""
  "rejectreason": ""
}
```

Failure Scenarios:

- If leave Id is not passed in query param

Sample Response:

Code: 400

```
{
  "statusCode": 400,
  "timestamp": "2026-06-08T10:46:27.442Z",
  "message": {
    "statusCode": 400,
    "timestamp": "2026-06-08T10:46:27.441Z",
    "message": {
      "message": [
        "leaveld must be a string",
        "leaveld should not be empty"
      ],
      "error": "Bad Request",
      "statusCode": 400
    }
  }
}
```

- **Incorrect leave Id passed, that does not exist**

Sample Response:

Code: 400

```
{
  "statusCode": 400,
  "timestamp": "2026-06-08T10:49:22.473Z",
  "message": {
    "statusCode": 400,
    "timestamp": "2026-06-08T10:49:22.467Z",
    "message": {
      "message": "Action forbidden: Request not found, already processed, or
unauthorized.",
      "error": "Bad Request",
      "statusCode": 400
    }
  }
}
```

- **Trying to reject a leaveld, that is not in PENDING state.**

Sample Response:

Code: 400

```
{
  "statusCode": 400,
  "timestamp": "2026-06-08T10:49:22.473Z",
  "message": {
```

```

        "statusCode": 400,
        "timestamp": "2026-06-08T10:49:22.467Z",
        "message": {
            "message": "Action forbidden: Request not found, already processed, or
unauthorized.",
            "error": "Bad Request",
            "statusCode": 400
        }
    }
}

```

- **If the user trying to reject the leave and does not have role as MANAGER**

Sample Response:
Code: 403

```

{
    "statusCode": 403,
    "timestamp": "2026-06-08T08:44:06.029Z",
    "message": {
        "statusCode": 403,
        "timestamp": "2026-06-08T08:44:06.027Z",
        "message": {
            "message": "You do not have the required role",
            "error": "Forbidden",
            "statusCode": 403
        }
    }
}

```

- **Wrong/Expired Jwt**

Code: 401
Sample Response:

```

{
    "statusCode": 401,
    "timestamp": "2026-06-08T08:07:28.459Z",
    "message": {
        "statusCode": 401,
        "timestamp": "2026-06-08T08:07:28.455Z",
        "message": {
            "message": "Unauthorized",
            "statusCode": 401
        }
    }
}

```

- **If reason is more than 100 characters**

Sample Response:

```
{
  "statusCode": 400,
  "timestamp": "2026-06-08T13:03:06.979Z",
  "message": {
    "statusCode": 400,
    "timestamp": "2026-06-08T13:03:06.974Z",
    "message": {
      "message": [
        "Reason must be under 100 characters"
      ],
      "error": "Bad Request",
      "statusCode": 400
    }
  }
}
```

8] **Endpoint:** leave-manager/leaves/employees/cancel

Method: POST

Query Param:

- leaveld (mandatory)

E.g.

leave-manager/leaves/employees/cancel?leaveld=cmq4yc6ns000001pj1oilip5t

Request Header:

Authorization: Bearer JWT_TOKEN

Allowed Roles: EMPLOYEE, MANAGER

Success Response:

```
{
  "id": LEAVE_ID,
  "employeeId": APPLICANT_ID,
  "type": SICK | CASUAL | PRIVILEGE,
  "startDate": start_date_of_leave,
  "endDate": end_date_of_leave,
  "status": CANCELLED,
  "reportingManagerId": reporting_manger_id
  "applyreason": ""
  "rejectreason": ""
}
```

```
}
```

Code: 200

Sample Response:

```
{
  "id": "cmq4yc6ns000001pj1oilip5t",
  "employeeId": "EMP-001",
  "type": "SICK",
  "startDate": "2026-06-08T00:00:00.000Z",
  "endDate": "2026-06-08T00:00:00.000Z",
  "status": "CANCELLED",
  "reportingManagerId": "MGR-002"
  "applyreason": ""
  "rejectreason": ""
}
```

Failure Scenarios:

- **If leave Id is not passed in query param**

Sample Response:

Code: 400

```
{
  "statusCode": 400,
  "timestamp": "2026-06-08T10:46:27.442Z",
  "message": {
    "statusCode": 400,
    "timestamp": "2026-06-08T10:46:27.441Z",
    "message": {
      "message": [
        "leaveld must be a string",
        "leaveld should not be empty"
      ],
      "error": "Bad Request",
      "statusCode": 400
    }
  }
}
```

- **Incorrect leave Id passed, that does not exist**

Sample Response:

Code: 400

```
{
  "statusCode": 400,
  "timestamp": "2026-06-08T10:49:22.473Z",
  "message": {
    "statusCode": 400,
    "timestamp": "2026-06-08T10:49:22.467Z",
    "message": {
      "message": "Action forbidden: Request not found, already processed, or
unauthorized.",
      "error": "Bad Request",
      "statusCode": 400
    }
  }
}
```

- **Trying to cancel a leaveld, that is not in PENDING state.**

Sample Response:
Code: 400

```
{
  "statusCode": 400,
  "timestamp": "2026-06-08T10:49:22.473Z",
  "message": {
    "statusCode": 400,
    "timestamp": "2026-06-08T10:49:22.467Z",
    "message": {
      "message": "Action forbidden: Request not found, already processed, or
unauthorized.",
      "error": "Bad Request",
      "statusCode": 400
    }
  }
}
```

- **If the user is trying to cancel the leave applied by someone else.**

Sample Response:
Code: 403

```
{
  "statusCode": 403,
  "timestamp": "2026-06-08T08:44:06.029Z",
  "message": {
    "statusCode": 403,
    "timestamp": "2026-06-08T08:44:06.027Z",
    "message": {
      "message": "You do not have the required role",

```

```

        "error": "Forbidden",
        "statusCode": 403
      }
    }
  }
}

```

- **Wrong/Expired Jwt**

Code: 401

Sample Response:

```

{
  "statusCode": 401,
  "timestamp": "2026-06-08T08:07:28.459Z",
  "message": {
    "statusCode": 401,
    "timestamp": "2026-06-08T08:07:28.455Z",
    "message": {
      "message": "Unauthorized",
      "statusCode": 401
    }
  }
}

```

Notifications Microservice Endpoints

- 1] **MessagePattern:** 'health'
Sample Response:
 { msg: 'UP' };

-
- 2] **MessagePattern:** leave-apply'
Data to be passed:

```

{
  leaveld: string ,
  employeeId: string,
  employeeName: string,
  reportingManagerId: string,
  reportingManagerName: string,

```

```
    type: string,
    startDate: Date,
    endDate: Date,
    status: PENDING | CANCELLED | APPROVED | REJECTED,
    duration: ,
    reason: string
  }
```

Response:

```
{msg: 'Notification Sent!'}
```

3] **MessagePattern:** 'leave-approved'

Data to be passed:

```
{
  leaveld: string ,
  employeeId: string,
  employeeName: string,
  reportingManagerId: string,
  reportingManagerName: string,
  type: string,
  startDate: Date,
  endDate: Date,
  status: PENDING | CANCELLED | APPROVED | REJECTED,
  duration: ,
  reason: string
}
```

Response:

```
{msg: 'Notification Sent!'}
```

4] **MessagePattern:** 'leave-rejected'

Data to be passed:

```
{
  leaveld: string ,
```

```
    employeeld: string,  
    employeeName: string,  
    reportingManagerId: string,  
    reportingManagerName: string,  
    type: string,  
    startDate: Date,  
    endDate: Date,  
    status: PENDING | CANCELLED | APPROVED | REJECTED,  
    duration: ,  
    reason: string  
  }
```

Response:
{msg: 'Notification Sent!'}

5] **MessagePattern:** 'leave-cancelled'

Data to be passed:

```
{  
  leaveld: string ,  
  employeeld: string,  
  employeeName: string,  
  reportingManagerId: string,  
  reportingManagerName: string,  
  type: string,  
  startDate: Date,  
  endDate: Date,  
  status: PENDING | CANCELLED | APPROVED | REJECTED,  
  duration: ,  
  reason: string  
}
```

Response:
{msg: 'Notification Sent!'}
